BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Session #4:
Edge Detection & Line Detection

Dhruba Adhikary,
Dhruba.a@wilp.bits-pilani.ac.in

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely and prepared by the lead instructor Prof S P Vimal

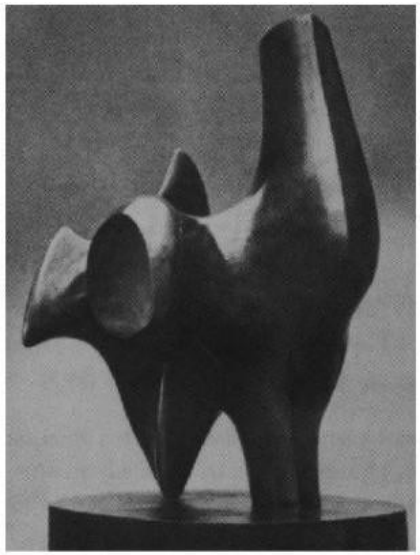
BITS Pilani
Pilani | Dubai | Goa | Hyderabad

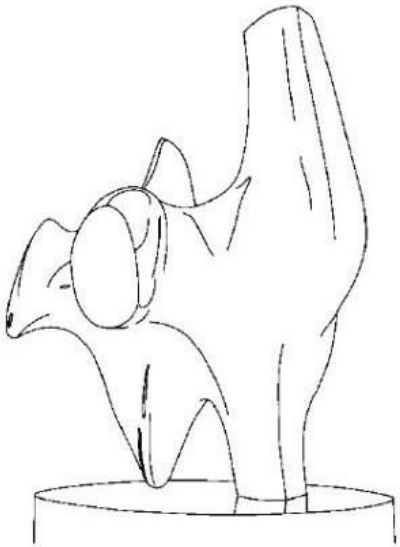
Topics

- Edge Models & Approaches to edge detection
- Hough Transform - Detecting Lines

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely

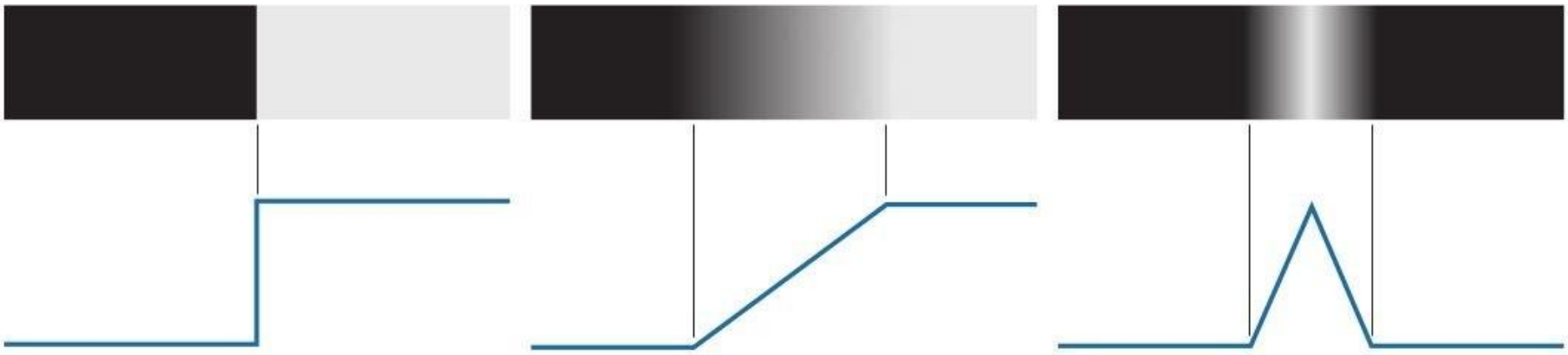
Edge Detection





Task : Convert a 2D image into a set of curves (made of edges)

Edge Profiles

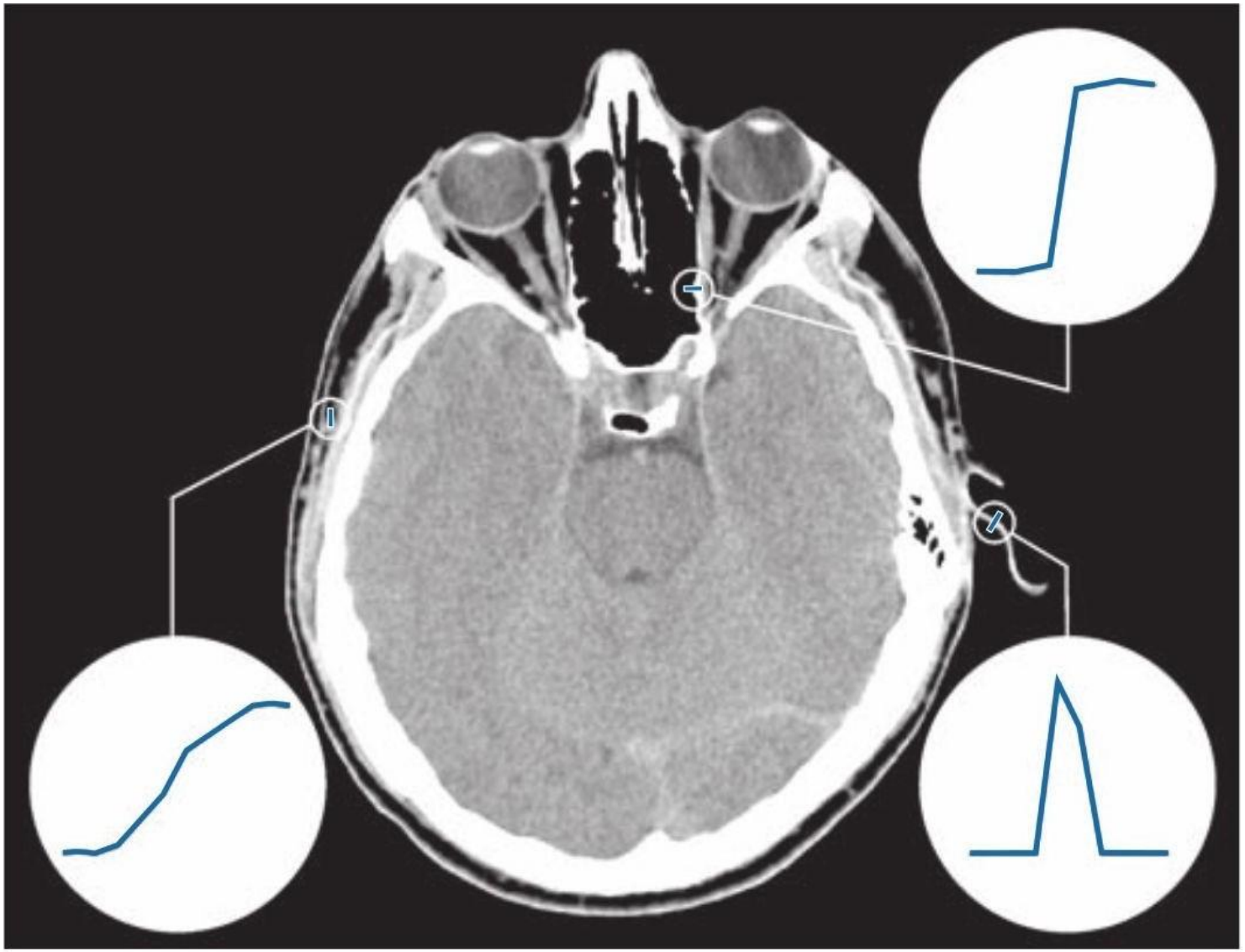


Step Edge

Ramp Edge

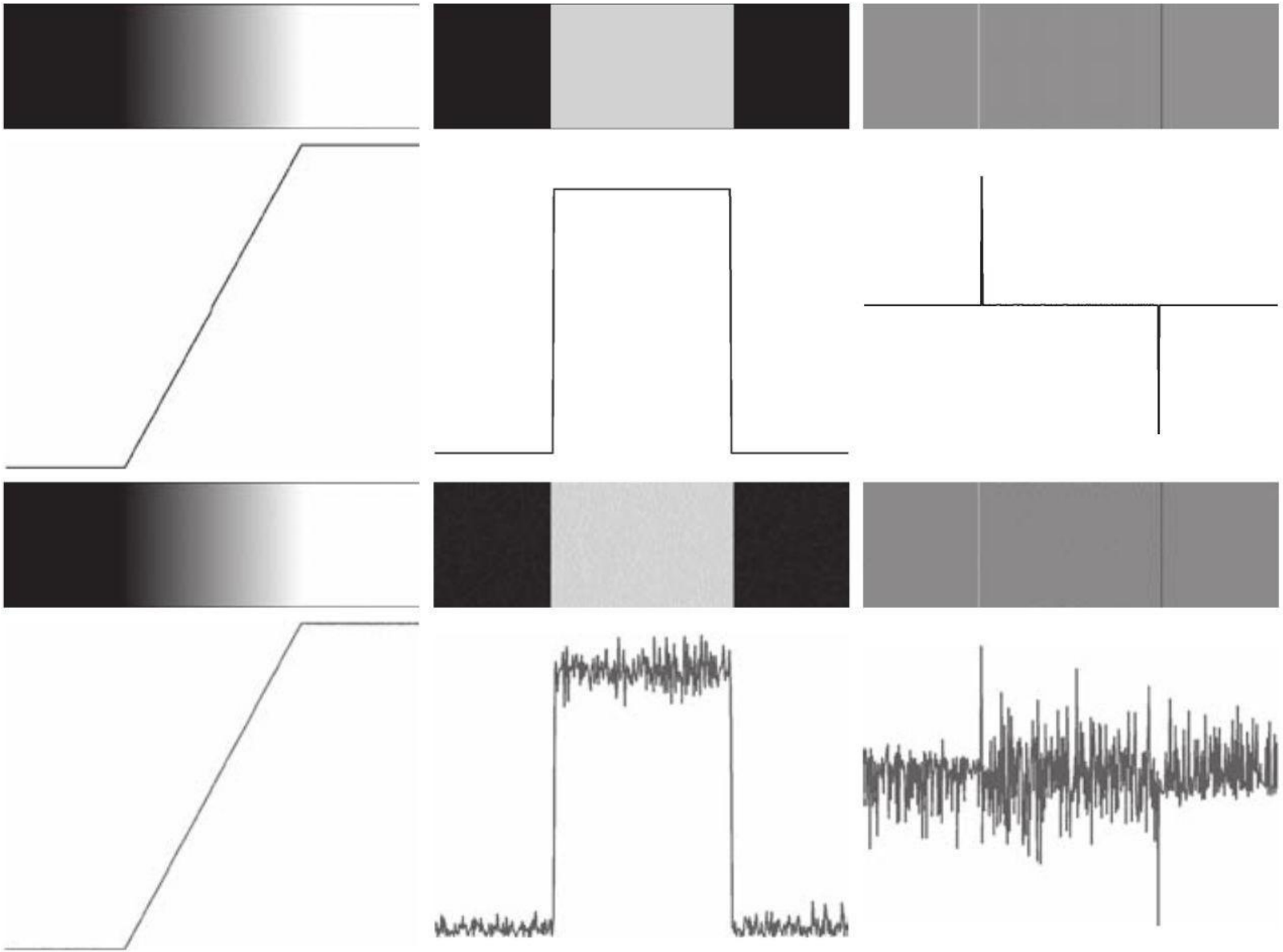
Roof Edge

Example



A 1508 1970 × image showing (zoomed) actual ramp (bottom, left), step (top, right), and roof edge profiles. The profiles are from dark to light, in the areas enclosed by the small circles. The ramp and step profiles span 9 pixels and 2 pixels, respectively. The base of the roof edge is 3 pixels. (Original image courtesy of Dr. David R. Pickens, Vanderbilt University.)

Edge Detection Approach



First column: 8-bit images with values in the range [,], 0 255 and intensity profiles of a ramp edge corrupted by Gaussian noise of zero mean and standard deviations of 0.0, 0.1, 1.0, and 10.0 intensity levels, respectively. Second column: First-derivative images and intensity profiles. Third column: Second-derivative images and intensity profiles.

Edge Detection Approach

1. Image smoothing for noise reduction.

2. Detection of edge points. As mentioned earlier, this is a local operation that extracts from an image all points that are potential edge-point candidates.

3. Edge localization. The objective of this step is to select from the candidate points only the points that are members of the set of points comprising an edge.

a b
c

FIGURE 10.2
(a) Image.
(b) Horizontal intensity profile that includes the isolated point indicated by the arrow.
(c) Subsampled profile; the dashes were added for clarity. The numbers in the boxes are the intensity values of the dots shown in the profile. The derivatives were obtained using Eqs. (10-4) for the first derivative and Eq. (10-7) for the second.

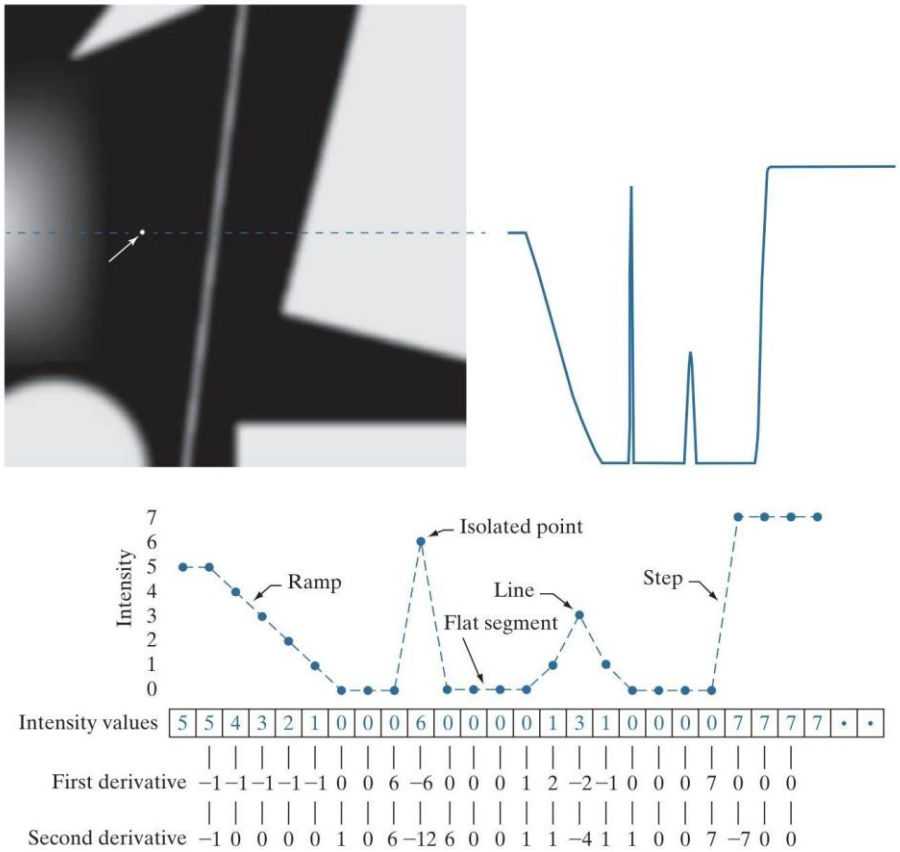


Image credit Digital Image Processing, 4th Ed, Rafael C. Gonzalez & Richard E. Woods

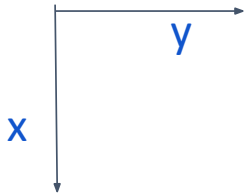
Gradient and it's Computation

- How can we differentiate a *digital* image F[x,y]?
  - Option 1: reconstruct a continuous image, *f,* then compute the derivative
  - Option 2: take discrete derivative (finite difference)

$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

Gradient and it's Computation

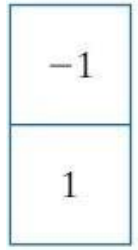


- How can we differentiate a *digital* image F[x,y]?
  - Option 1: reconstruct a continuous image, *f,* then compute the derivative
  - Option 2: take discrete derivative (finite difference)

$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

(a)



(b)

Gradient and it's Computation

Compute partial derivatives ∂f/∂x and ∂f/∂y at every pixel location in the image

$$\nabla f(x,y) \equiv \text{grad}[f(x,y)] \equiv \begin{bmatrix} g_x(x,y) \\ g_y(x,y) \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f(x,y)}{\partial x} \\ \dfrac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

Points in the direction of maximum rate of change of f at (x, y)

$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$

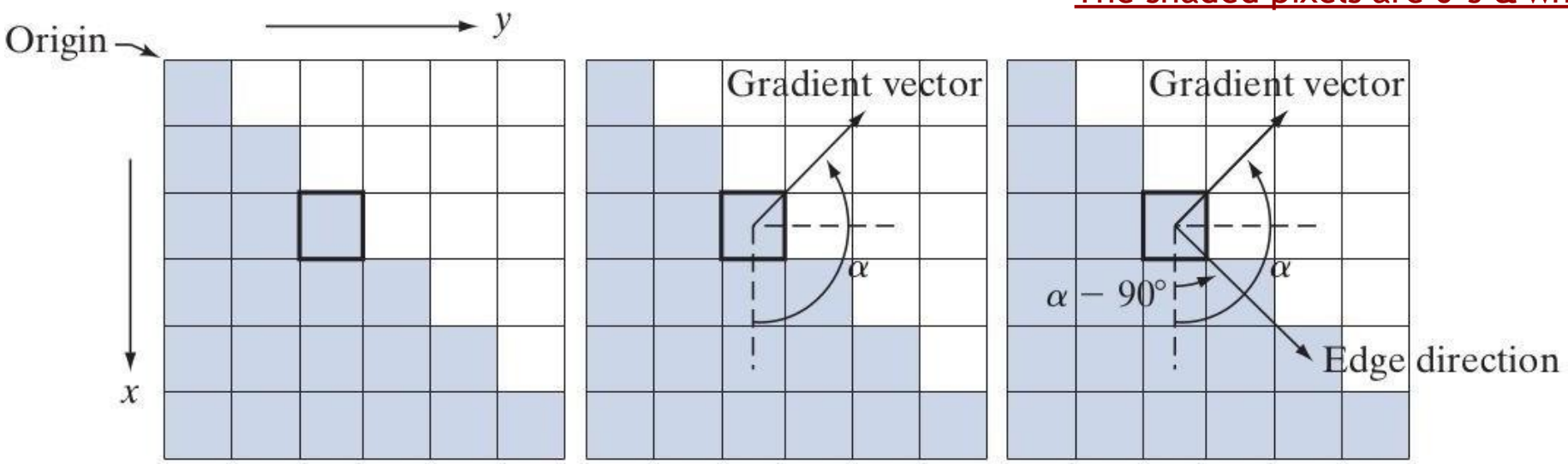
Direction of gradient vector at (x, y)

$$M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$$

Value of the rate of change in the direction of the gradient vector at (x, y)

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's

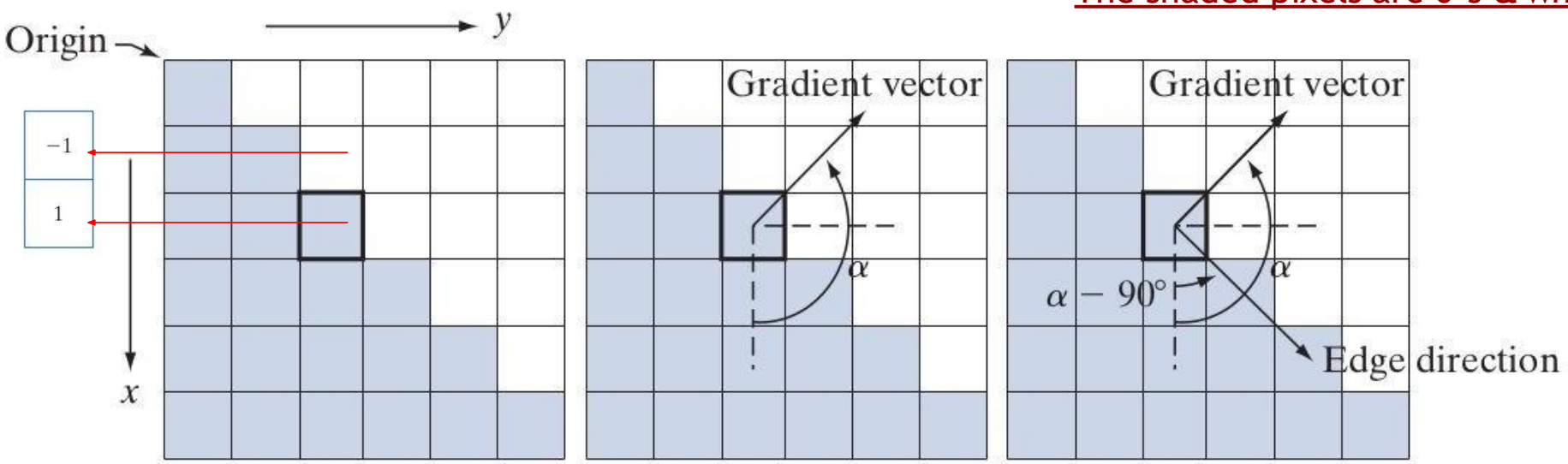


$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

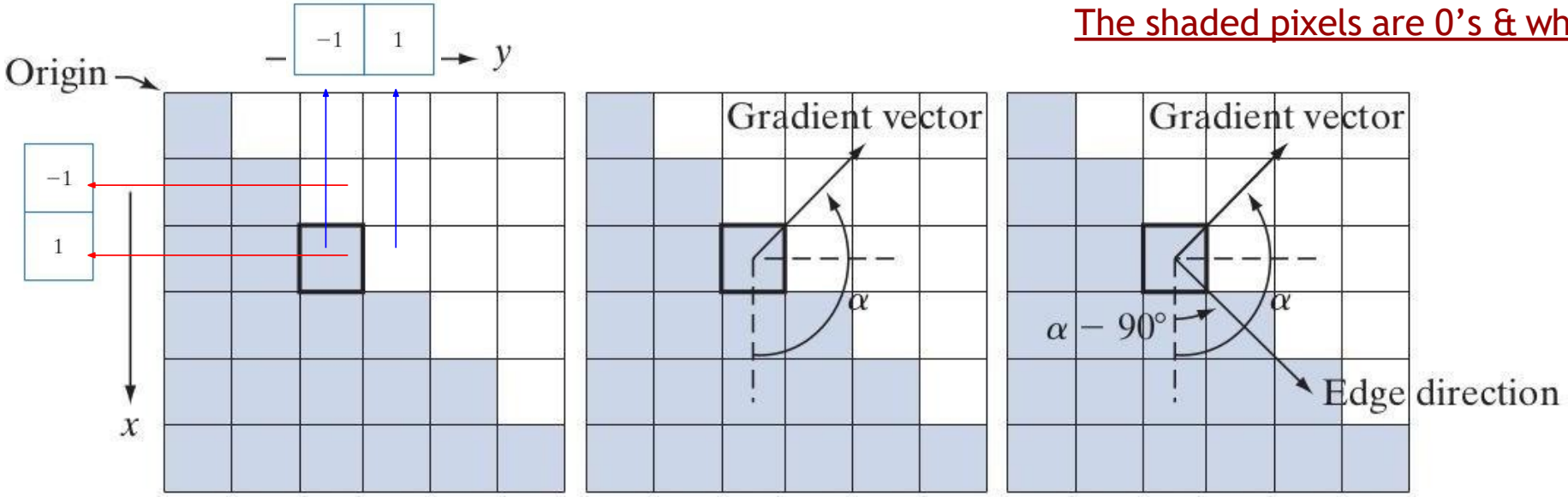
Subtract pixels in the top row of the neighborhood from the pixels in the bottom row → partial derivative in the x-direction

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

Subtract pixels in the left column from the pixels in the right column of the neighborhood → partial derivative in the y-direction

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



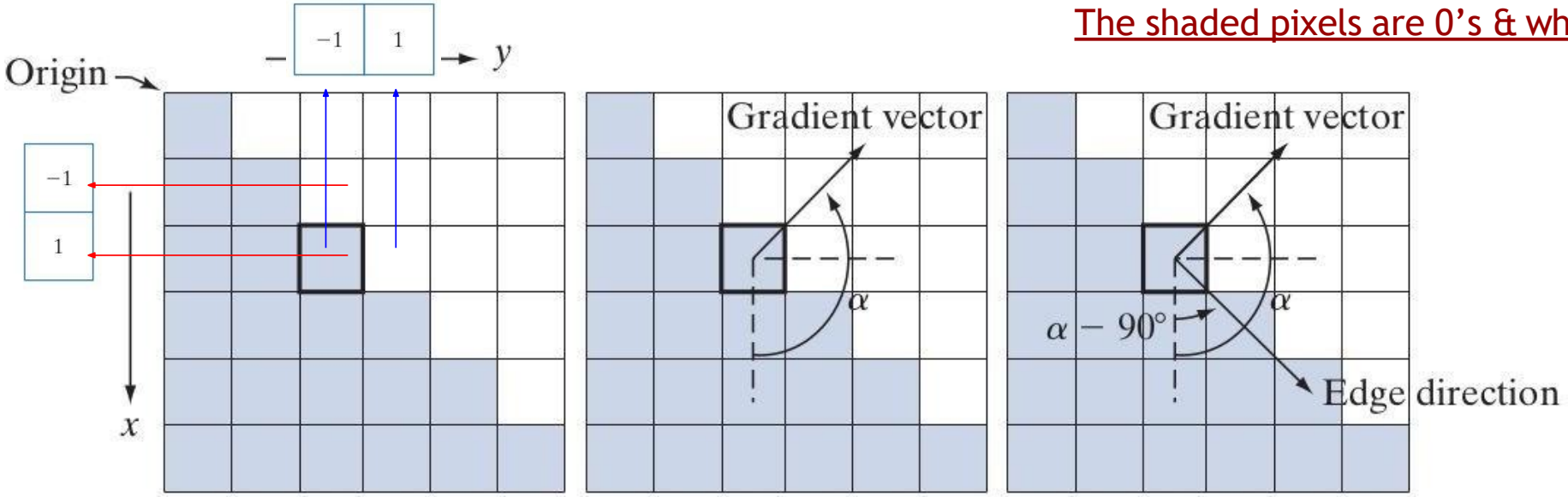
$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y) \quad = -2$$

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y) \quad = 2$$

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



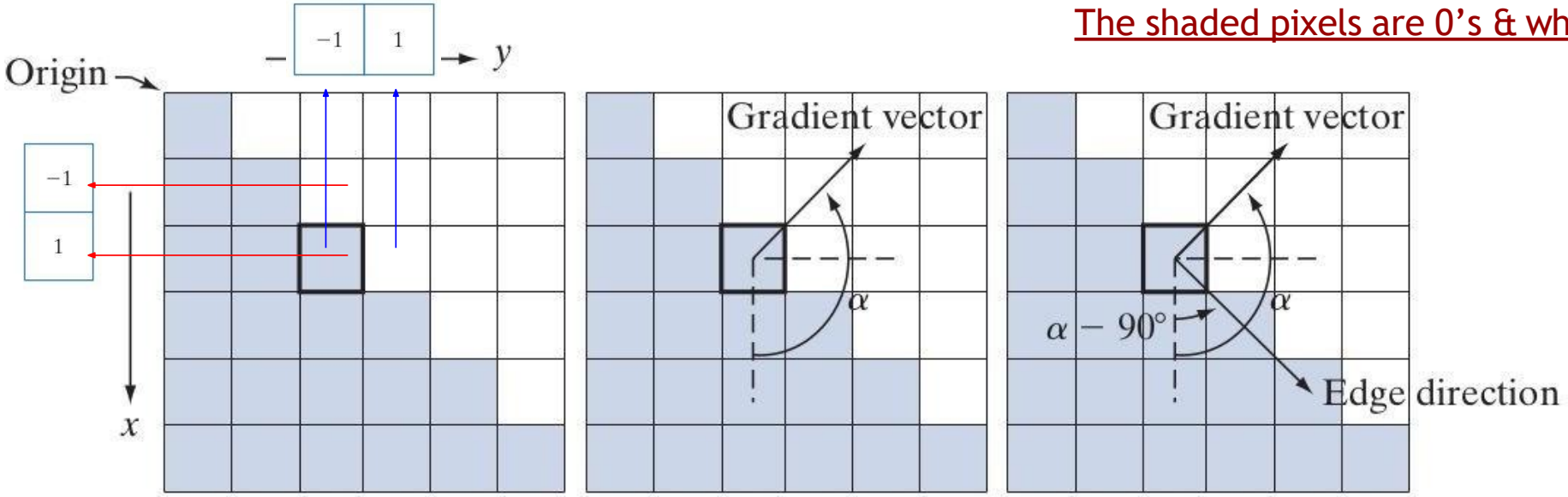
$$\|\nabla f\| = 2\sqrt{2}$$

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

$$\alpha = \tan^{-1}\left(g_y / g_x\right) = -45° \quad = 135°$$

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



Direction of an edge at a point is orthogonal to the gradient vector at that point ⇒ Direction of Edge is

$$\alpha - 90° = 135° - 90° = 45°$$

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

$$\alpha = \tan^{-1}\left(g_y / g_x\right) = -45° \quad = 135°$$

Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Prewitt's operators

−1	−1	−1
0	0	0
1	1	1

−1	0	1
−1	0	1
−1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Sobel's operators

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Gradient and it's Computation - How to use?

(1) Use any of the gradient operators to obtain g_x and g_y at every pixel location

(2) Approximate gradient at each of the pixel locations as below:

$$M(x,y) \approx |g_x| + |g_y|$$

Gradient and it's Computation - Example

a b
c d

(a) Image of size 834 × 1114 pixels, with intensity values scaled to [0, 1].

(b) $|g_x|$, the component of gradient obtained using the Sobel kernel

(c) $|g_y|$

(d) The gradient (using sum of absolute values)

Gradient and it's Computation - Example

a b
c d

Same sequence as prev. fig. but with the original image smoothed using a 5 × 5 averaging kernel prior to edge detection.

Canny Edge Detection

A powerful Edge Detection Approach,
Deal with iit in the Next Class

Image Sharpening - Next Class

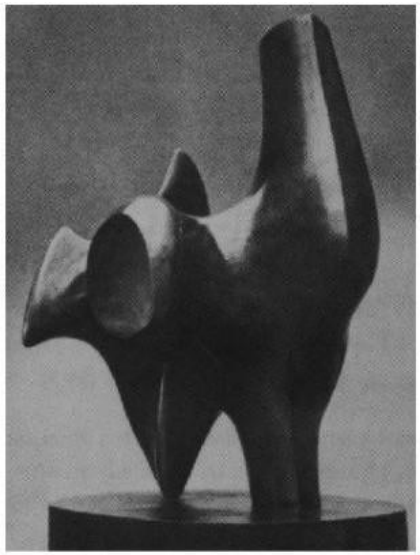
Edge Detection

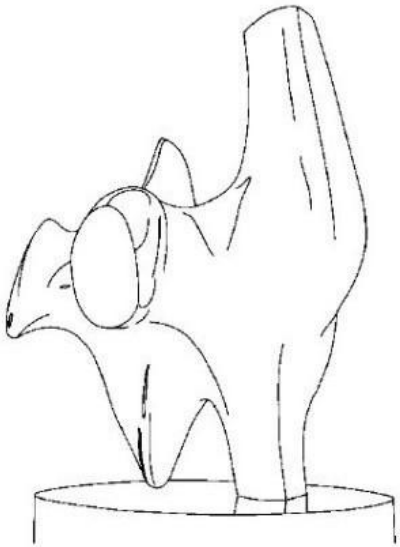
- Edges are pixels where *intensity (brightness)* changes abruptly
- Gradient's (along x and y direction) are used to describe the direction of the largest growth of the image function
  - Derivatives are approximated by differences

- <u>Objectives of Edge Detection</u>

  - Good Detection. Filter responds to edge, not noise
  - Good Localization: detected edge near true edge.
  - Single Response: only one point for each true edge point

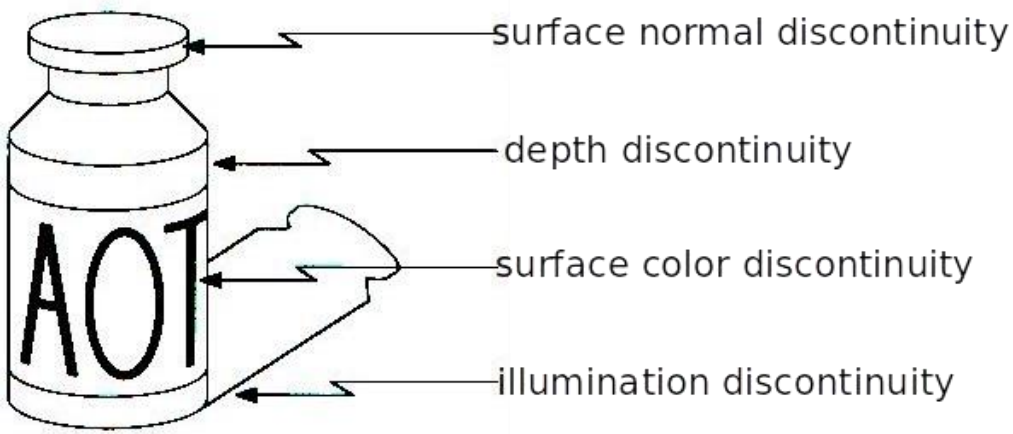
Edge Detection





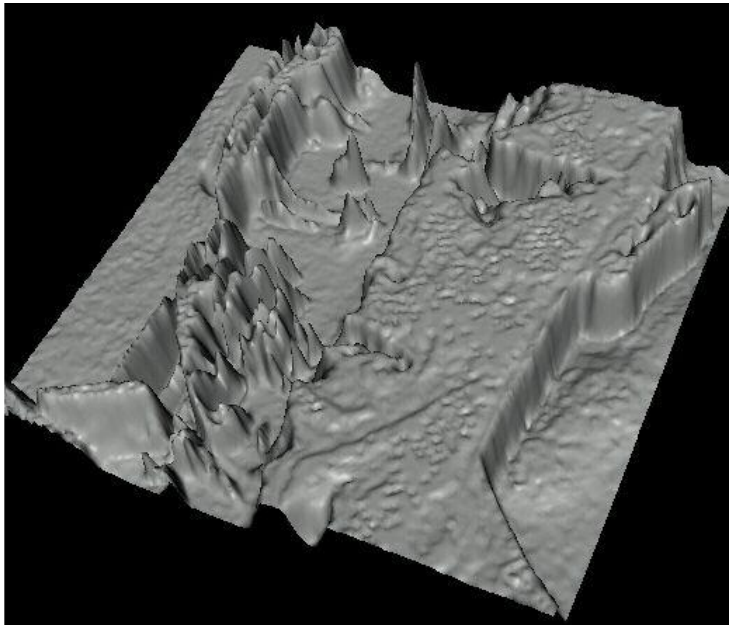
Task : Convert a 2D image into a set of curves (made of edges)

Edge Detection



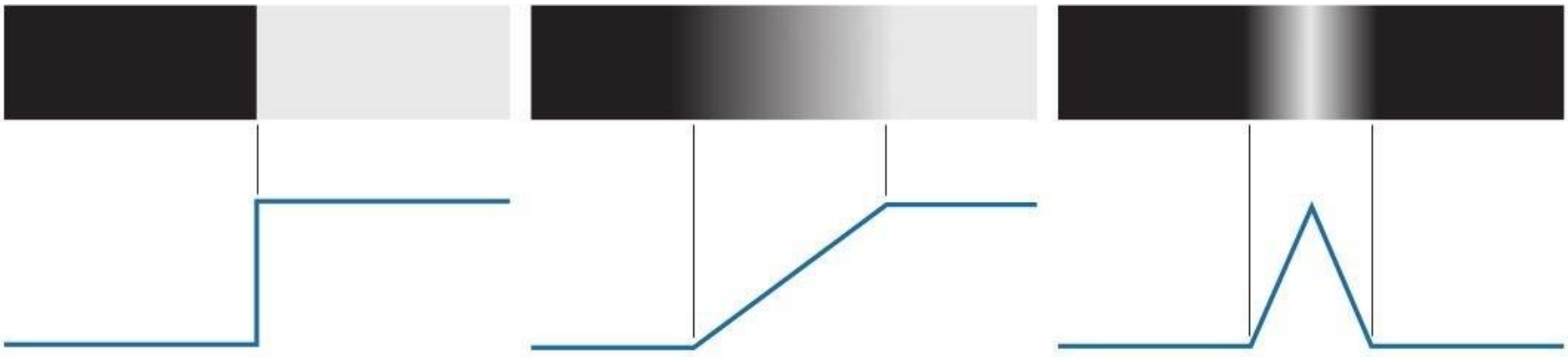
Edges are caused by a variety of factors

Edge Detection



Treating images as functions, edges look like steep cliffs in the visualization

Recall: Edge Profiles



Step Edge

Ramp Edge

Roof Edge

Recall: Edge Detector

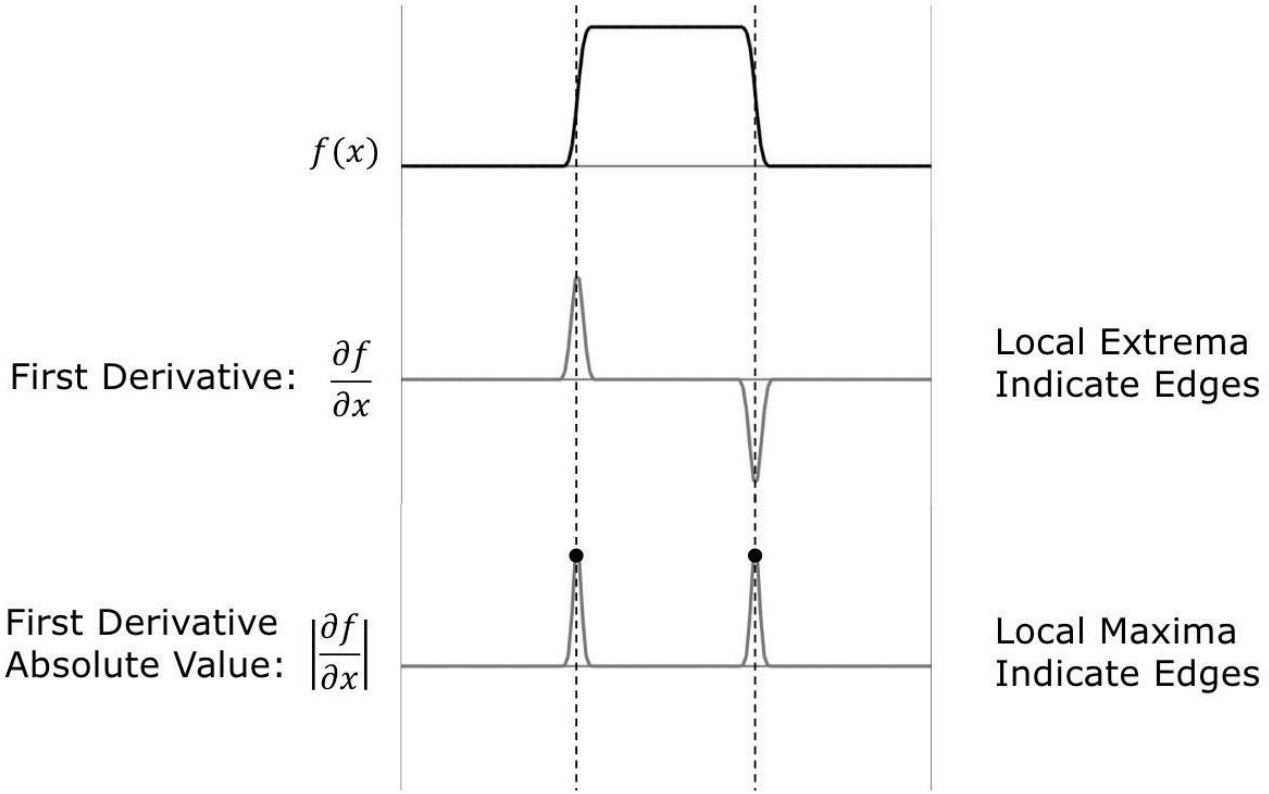
We want an Edge Operator that produces:

- Edge Position
- Edge Magnitude (Strength)
- Edge Orientation (Direction)

Performance Requirements:

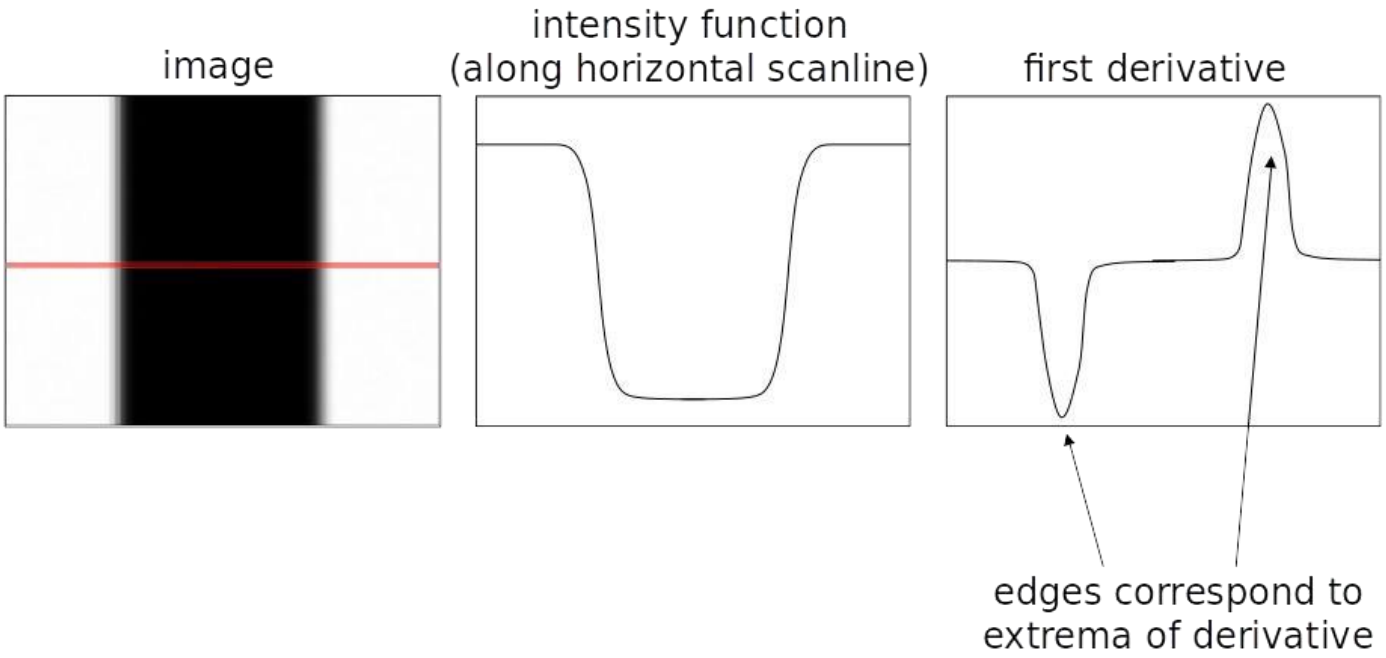
- High Detection Rate
- Good Localization
- Low Noise Sensitivity

Recall: Edge detection using 1st Derivative



Provides Both Location and Strength of an Edge

Recall: Edge Profiles



An edge is a place of rapid change in the image intensity function

Recall: Gradient and it's Computation

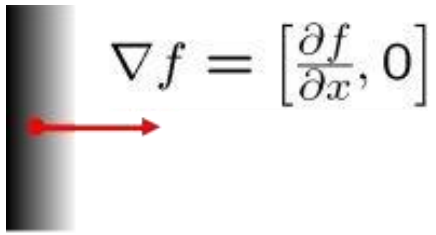
- How can we differentiate a *digital* image F[x,y]?
  - Option 1: reconstruct a continuous image, *f,* then compute the derivative
  - Option 2: take discrete derivative (finite difference)

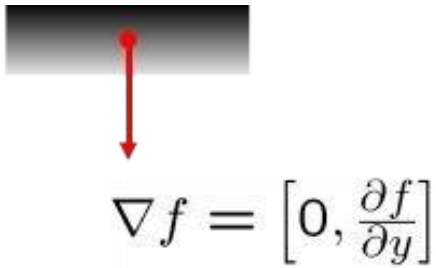
$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

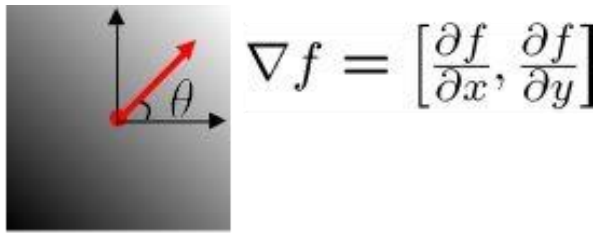
$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

Recall: Gradient's role in Detecting Edge

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$
- The gradient points in the *direction of most rapid increase in intensity*







- The edge strength is given by the gradient magnitude: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$
- The gradient direction is given by: $\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$

Comparing Gradient operators

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	0 1 -1 0	-1 0 1 -1 0 1 -1 0 1	-1 0 1 -2 0 2 -1 0 1	-1 -2 0 2 1 -2 -3 0 3 2 -3 -5 0 5 3 -2 -3 0 3 2 -1 -2 0 2 1
$\frac{\partial I}{\partial y}$	1 0 0 -1	1 1 1 0 0 0 -1 -1 -1	1 2 1 0 0 0 -1 -2 -1	1 2 3 2 1 2 3 5 3 2 0 0 0 0 0 -2 -3 -5 -3 -2 -1 -2 -3 -2 -1

Good Localization
Noise Sensitive
Poor Detection

Poor Localization
Less Noise Sensitive
Good Detection

Slide: Shree K Nayar

Steps in Canny's Edge Detection

1. Smooth the input image with a Gaussian filter.
2. Compute the gradient magnitude and angle images.
3. Apply non-maxima suppression to the gradient magnitude image.
4. Use double thresholding and connectivity analysis to detect and link edges.

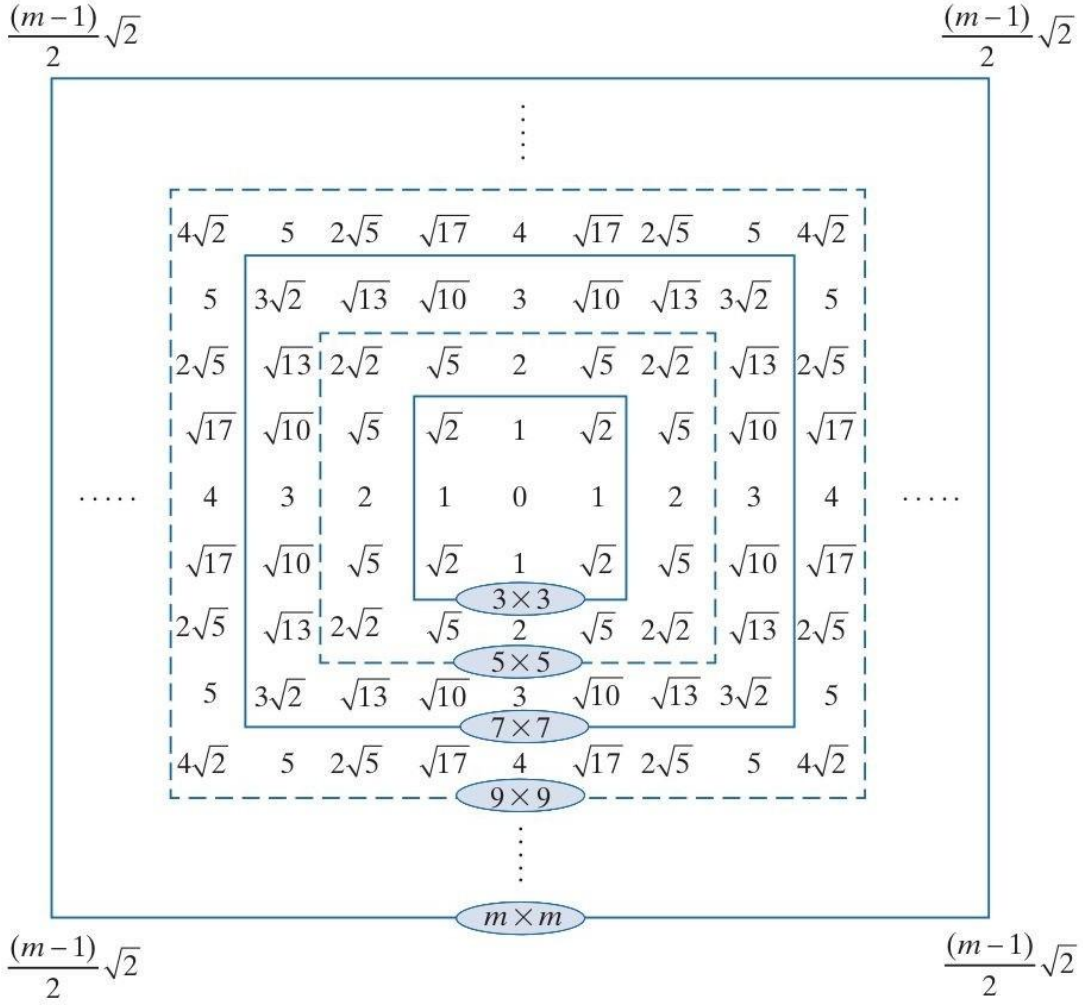
Recall: Gaussian Filters

Let s and r be position of a cell in a filter w.r.t the origin; r represent the distance of the cell from the center

$$r = [s^2 + t^2]^{1/2}$$

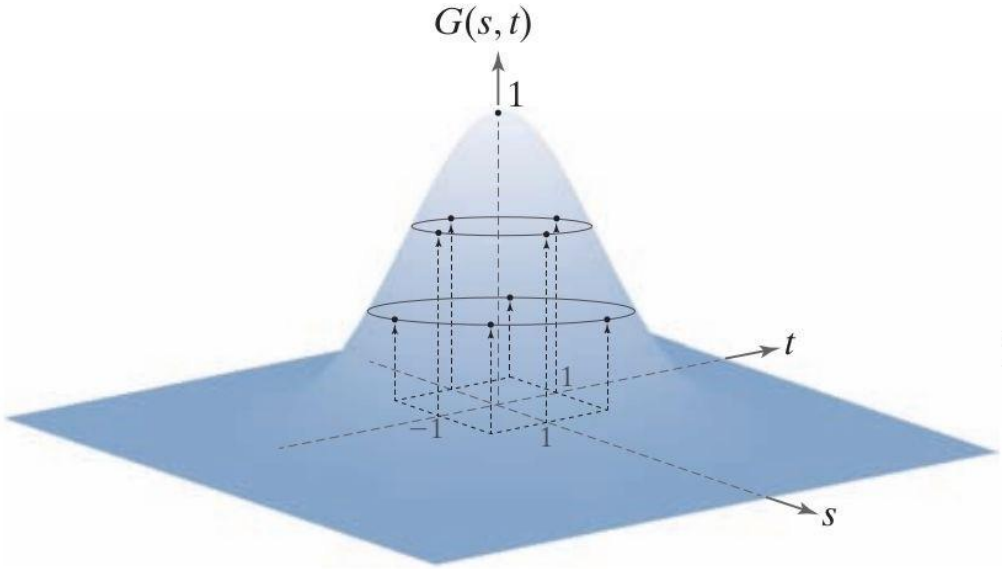
Form of Gaussian kernels is as below:

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$



Distances from the center for various sizes of square kernels.

Recall: Gaussian Filters



$$\frac{1}{4.8976} \times \begin{bmatrix} 0.3679 & 0.6065 & 0.3679 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.3679 & 0.6065 & 0.3679 \end{bmatrix}$$

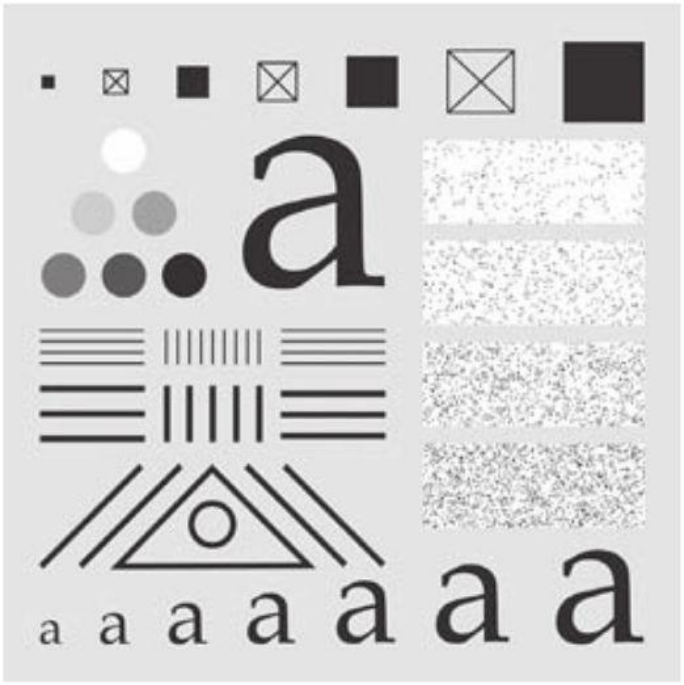
Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for K = 1 and σ= 1.

Resulting 3 × 3 kernel

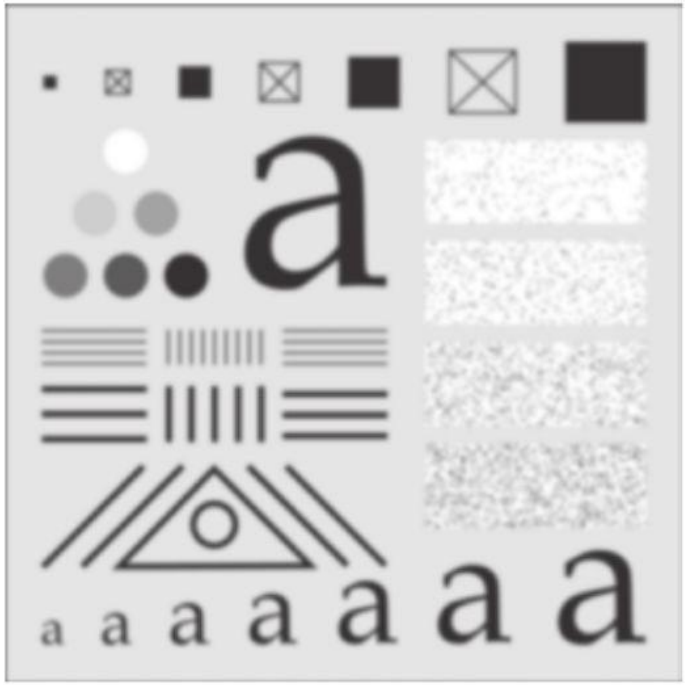
$$r = [s^2 + t^2]^{1/2}$$

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

Recall: Gaussian Filters



A test pattern of size 1024 × 1024



lowpass filtering the pattern with a Gaussian kernel of size 21 × 21, with σ=3.5, K=1

Result of using a kernel of size 43 × 43, with σ= 7, K=1

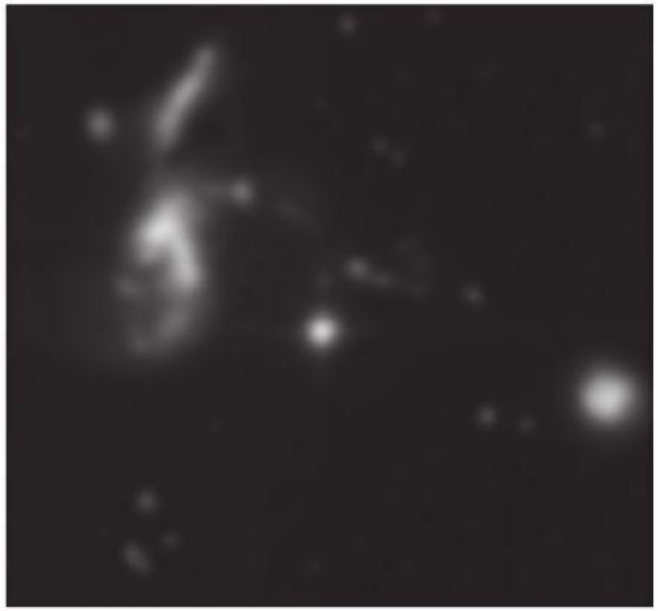
Recall: Gaussian Filters

Gaussian kernels of size 43 × 43, with $\sigma = 7$

Kernel of 85 × 85, with $\sigma = 7$

Difference image

Recall: Gaussian Filters - Example Usage



a b c

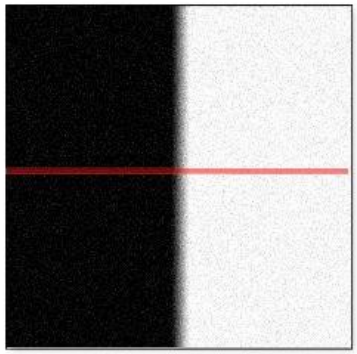
A 2566 × 2758 Hubble Telescope image of the Hickson Compact Group

Result of lowpass filtering with a Gaussian kernel.

Result of thresholding the filtered image (intensities were scaled to the range [0, 1])

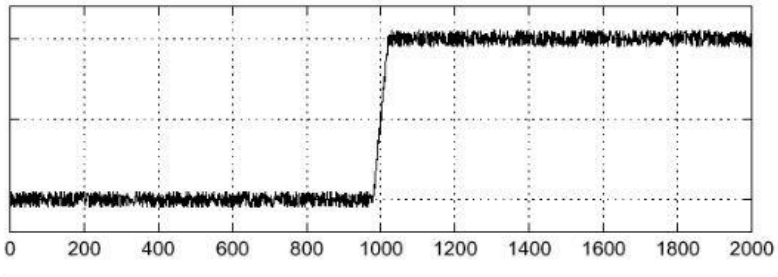
Effect of Noise

- Where is the edge?

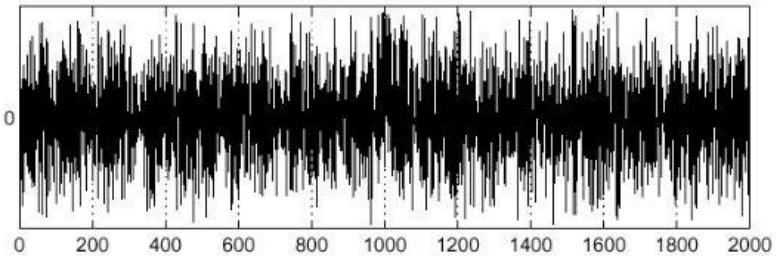


Noisy input image

$f(x)$

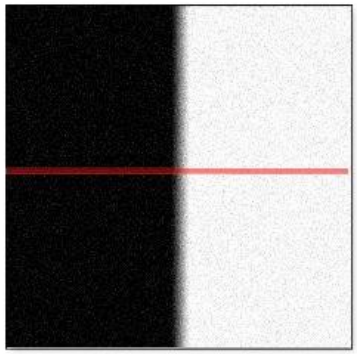


$\frac{d}{dx}f(x)$

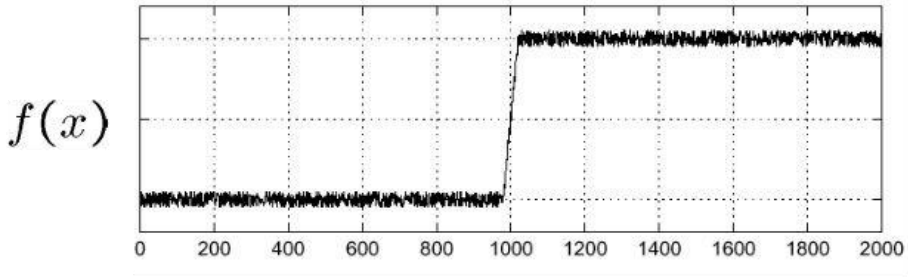


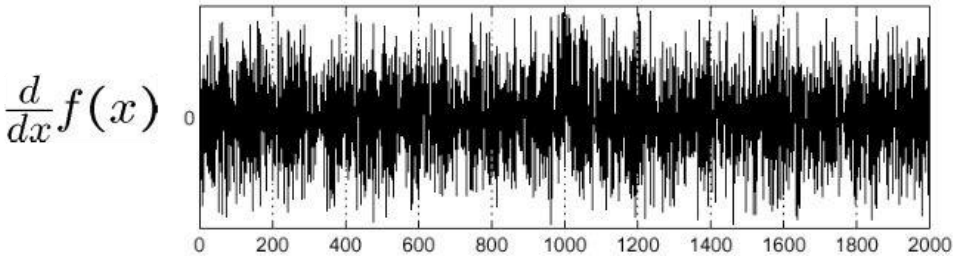
Effect of Noise

- Where is the edge?



Noisy input image





Step -1: Smooth the image with gaussian low-pass filter

Step -2: Computing Gradients (Magnitude and Direction)

Compute partial derivatives ∂f/∂x and ∂f/∂y at every pixel location in the image

$$\nabla f(x,y) \equiv \text{grad}[f(x,y)] \equiv \begin{bmatrix} g_x(x,y) \\ g_y(x,y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

Points in the direction of maximum rate of change of f at (x, y)

$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$

Direction of gradient vector at (x, y)

$$M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$$

Value of the rate of change in the direction of the gradient vector at (x, y)

Step -2: Computing Gradients (Magnitude and Direction)

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Prewitt's operators

−1	−1	−1
0	0	0
1	1	1

−1	0	1
−1	0	1
−1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Step -2: Computing Gradients (Magnitude and Direction)

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Sobel's operators

−1	−2	−1
0	0	0
1	2	1

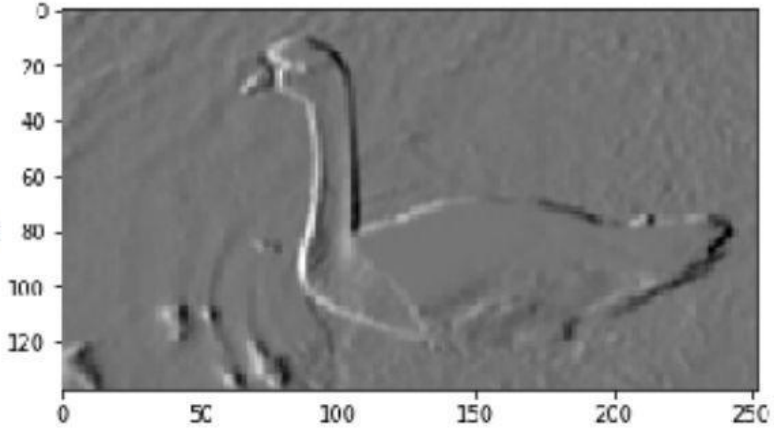
−1	0	1
−2	0	2
−1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

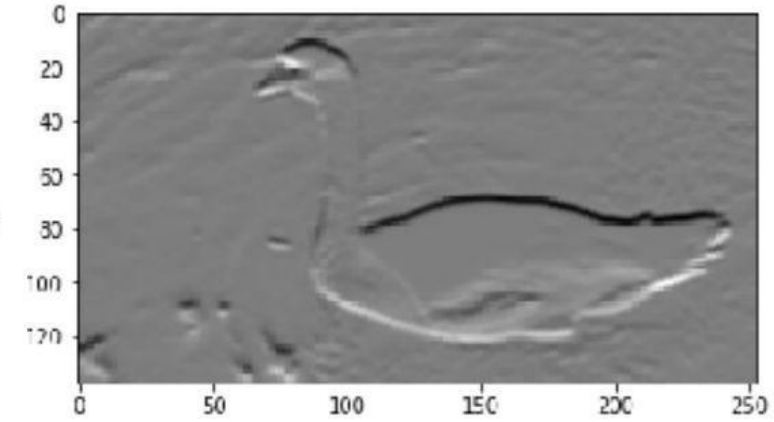
$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

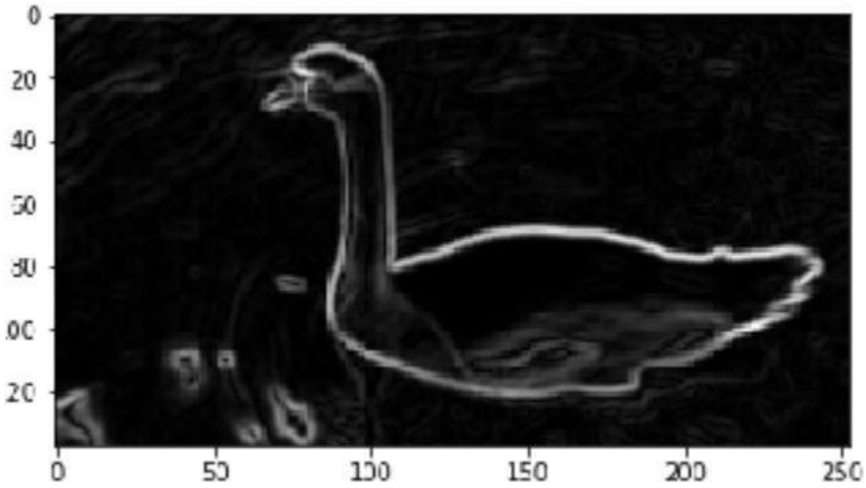
Step -2: Computing Gradients (Magnitude and Direction)

F_x



F_y

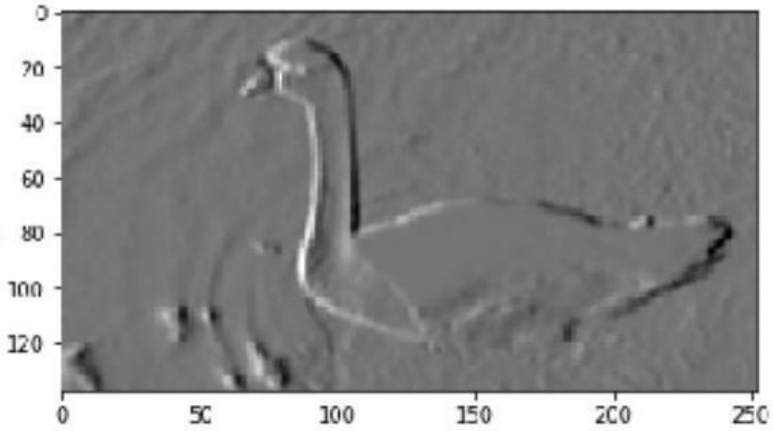


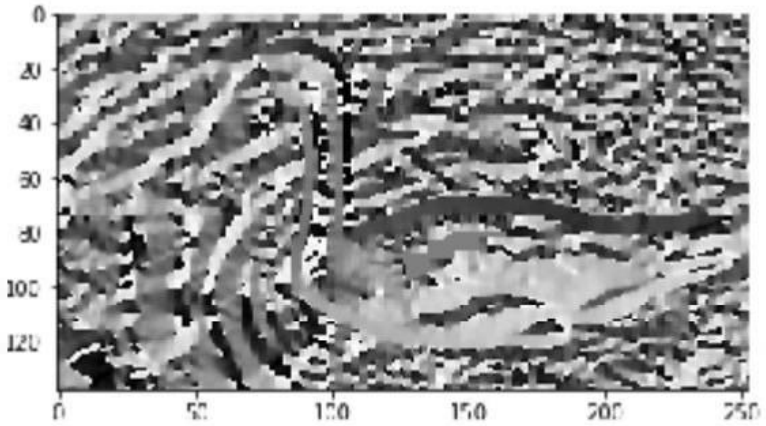


$$G = \sqrt{(F_x^2 + F_y^2)}$$

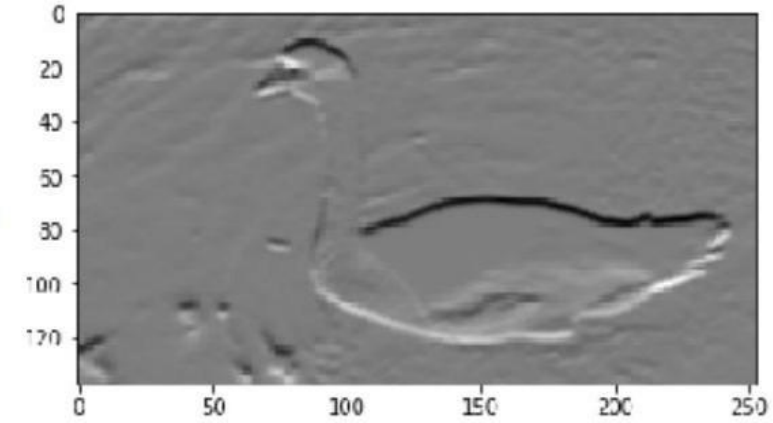
Step -2: Computing Gradients (Magnitude and Direction)

F_x



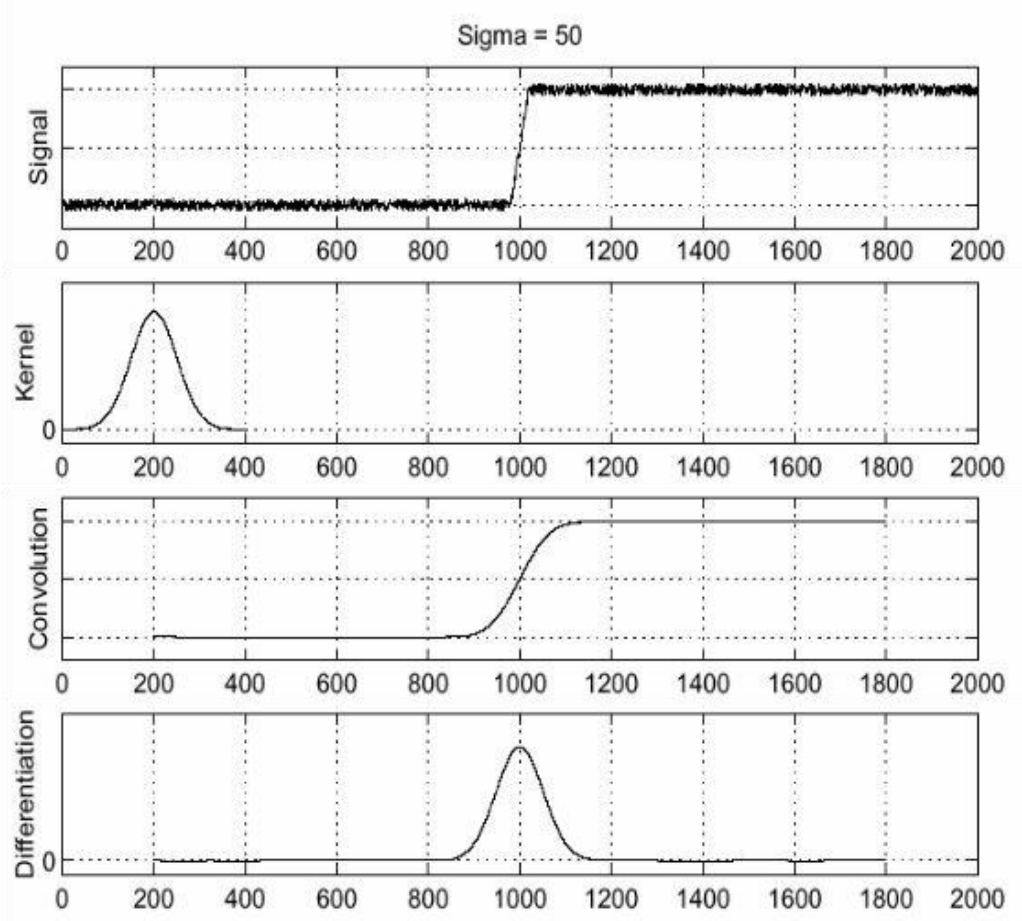


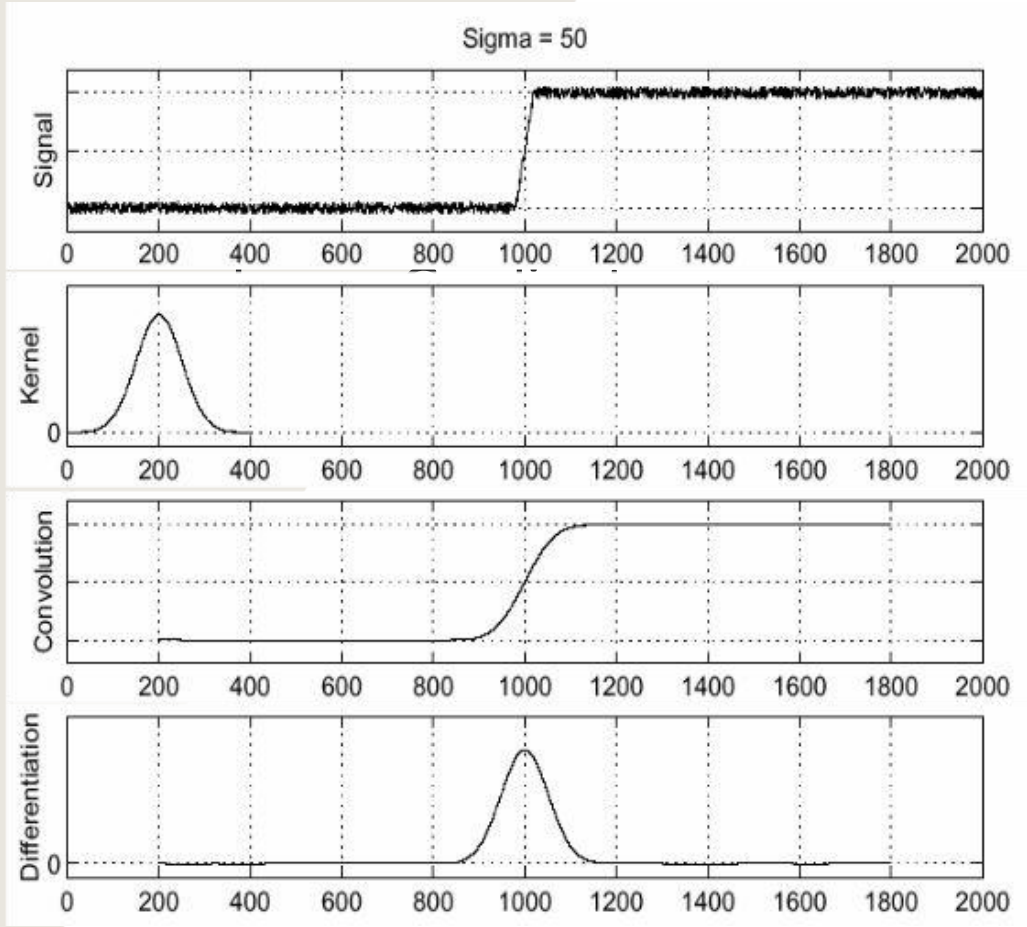
F_y



$$\theta = tan^{-1}\left(\frac{F_y}{F_x}\right)$$

(Aside) Step -2: Efficient Computation of Image Gradients



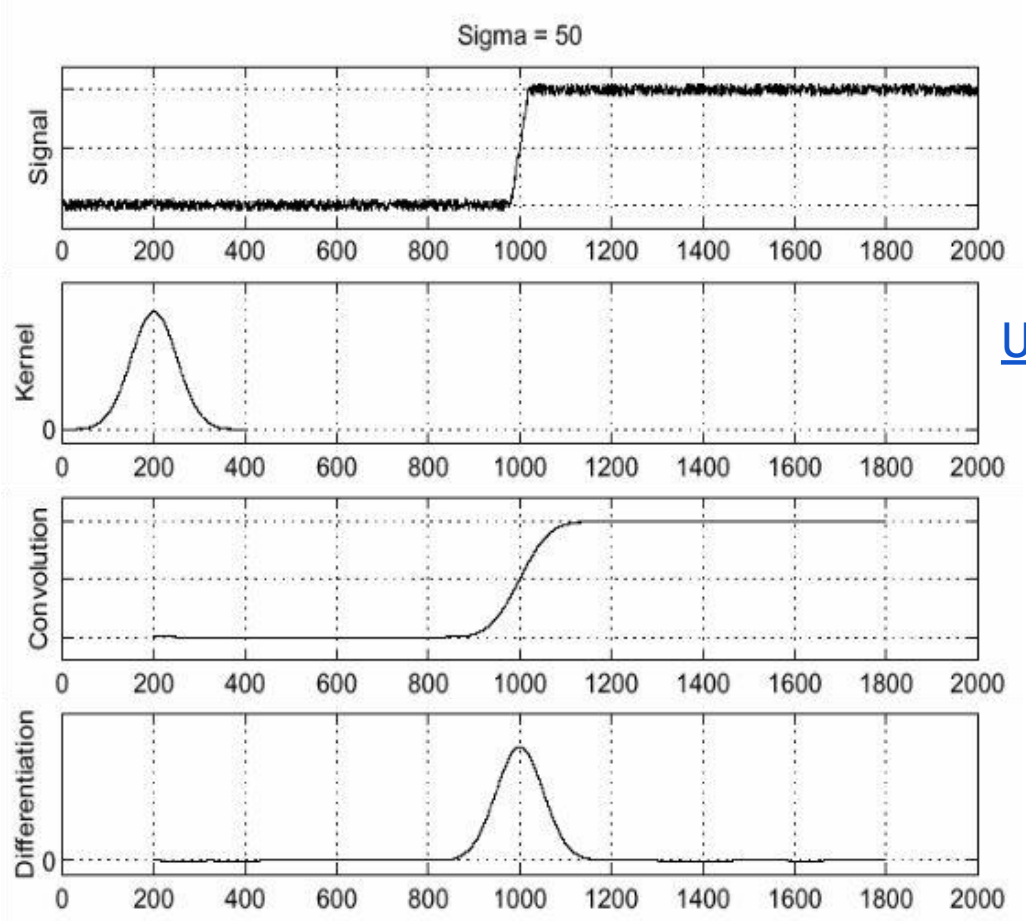


$\frac{d}{dx}(f * h)$

Objective is to compute $\frac{d}{dx}(f * h)$

ıtation of

(Aside) Step -2: Efficient Computation of Image Gradients



$$\frac{d}{dx}(f * h)$$

Objective is to compute $\frac{d}{dx}(f * h)$

Using associative property of convolution

$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

$$G_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$G'_\sigma(x) = \frac{d}{dx}G_\sigma(x) = -\frac{1}{\sigma}\left(\frac{x}{\sigma}\right)G_\sigma(x)$$

This saves us one operation

Ex: Step -1 and Step-2

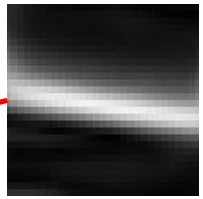
Original Image

Ex: Step -1 and Step-2

Smoothed Gradient Magnitude

Original Image

ep-2

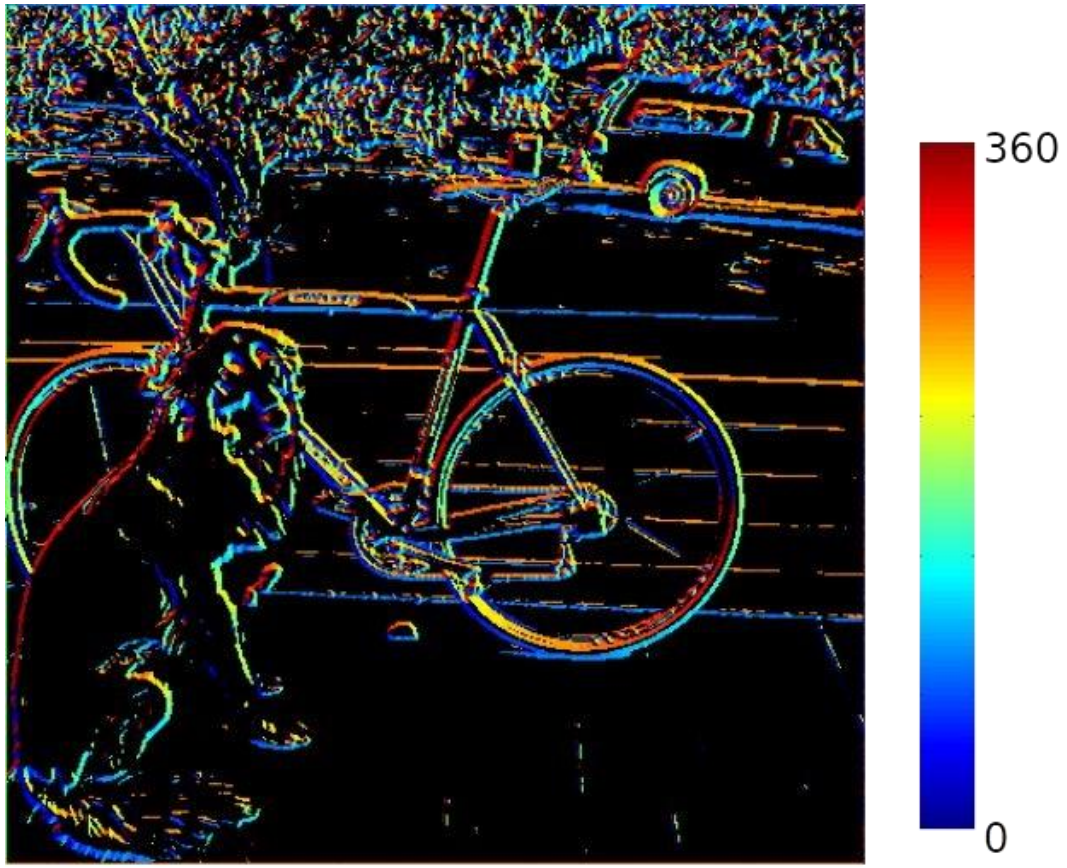


Where is the edge?

Smoothed Gradient Magnitude

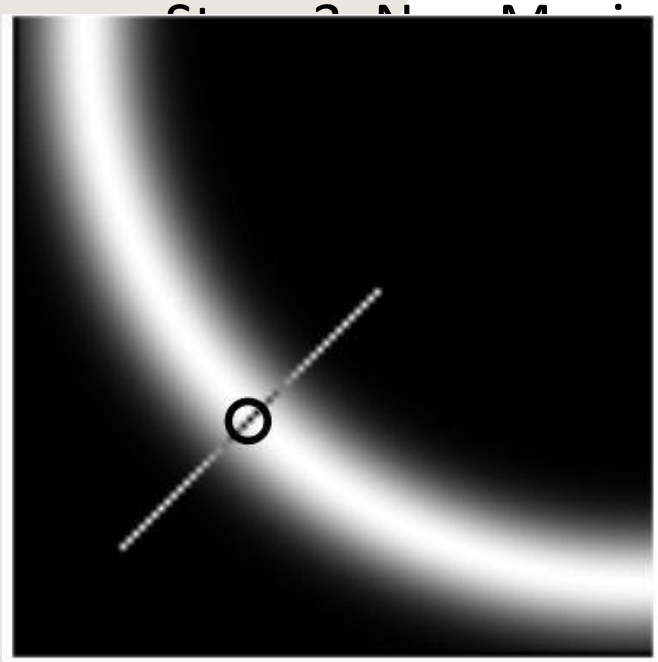
Ex: Step -1 and Step-2

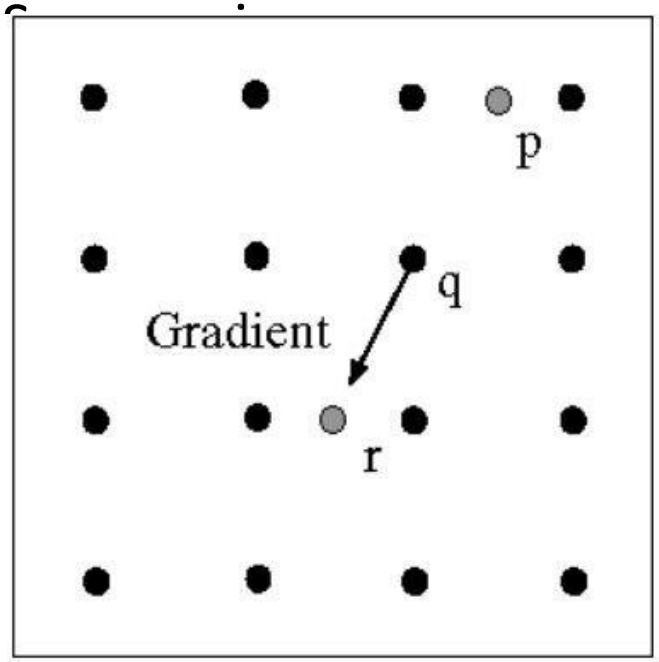
Smoothed Gradient Magnitude



Can we make use of angle to locate the edge better?

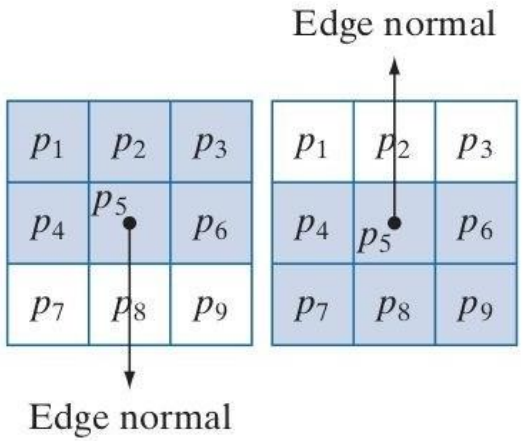
Check if pixel is local maximum along gradient direction

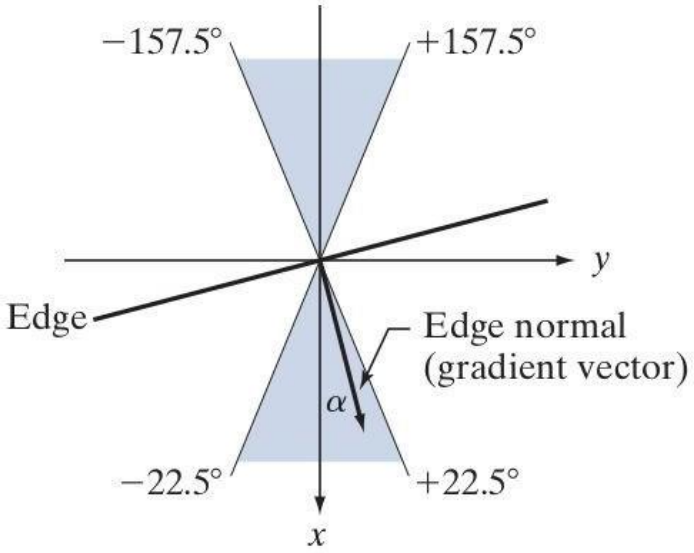


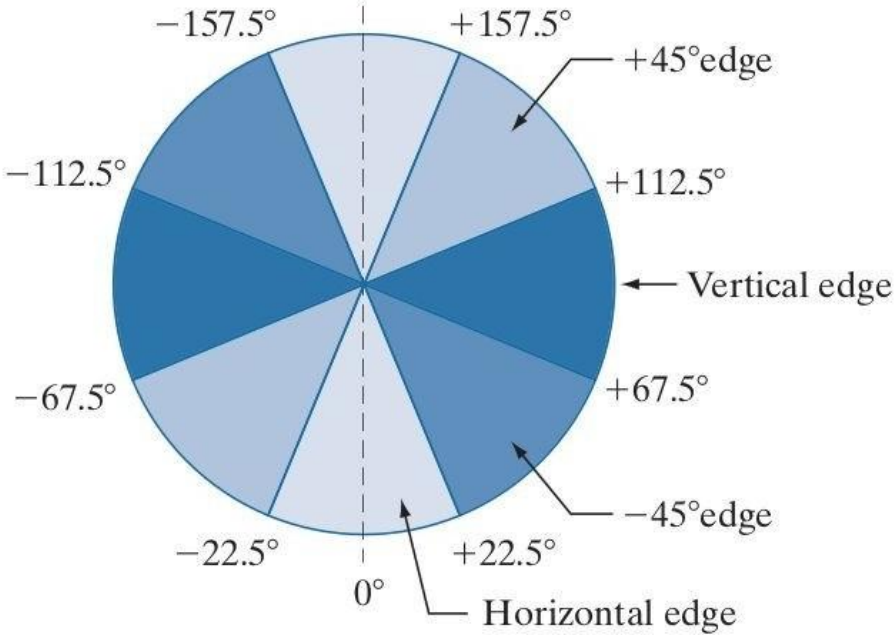


Step -3: Non-Maxima Suppression - Computation

Let d_1, d_2, d_3 and d_4 refers to horizontal, vertical, +45° and -45° edges.







Step -3: Non-Maxima Suppression - Computation

Let α be the matrix if gradient orientations

Let d_1, d_2, d_3 and d_4 refers to horizontal, vertical, +45° and -45° edges.

Consider a 3x3 neighborhood around $\alpha(x,y)$

Non-Maxima Scheme around $\alpha(x,y)$ is as below:

1. Find the direction d_k that is closest to $\alpha(x,y)$.
2. Let K denote the value of $\|\nabla f_s\|$ at (x,y). If K is less than the value of $\|\nabla f_s\|$ at one or both of the neighbors of point (x,y) along d_k, let $g_N(x,y) = 0$ (suppression); otherwise, let $g_N(x,y) = K$.

Repeat (1)and (2) for all values of x and y to get non-maxima suppressed image which is of the same size as original image.

Step -3: Non-Maxima Suppression - Computation

Before Non-Maxima Suppression

After Non-Maxima Suppression

Step -4: Double Thresholding and Edge Linking

Still some noise; Only need strong edges;

Called as *Hysteresis thresholding*

Use two thresholds T_L and T_H which results in two images;

Choose the ratio of high to low threshold in the range of 2:1 to 3:1.

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

Strong Edge Pixels & Valid Edge Pixels

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Weak Edge Pixels & we need to identify valid edge pixels from this set.

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

Strong Edge Pixels & Valid Edge Pixels

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Weak Edge Pixels & we need to identify valid edge pixels from this set.

Step -4: Double Thresholding and Edge Linking

Approach to Link Edges:

- Strong edges are edges!
- Weak edges are edges iff they connect to strong
- Look in some neighborhood (usually 8 closest)

Step -4: Double Thresholding and Edge Linking

(a) Locate the next unvisited edge pixel, p, in $g_{NH}(x, y)$.

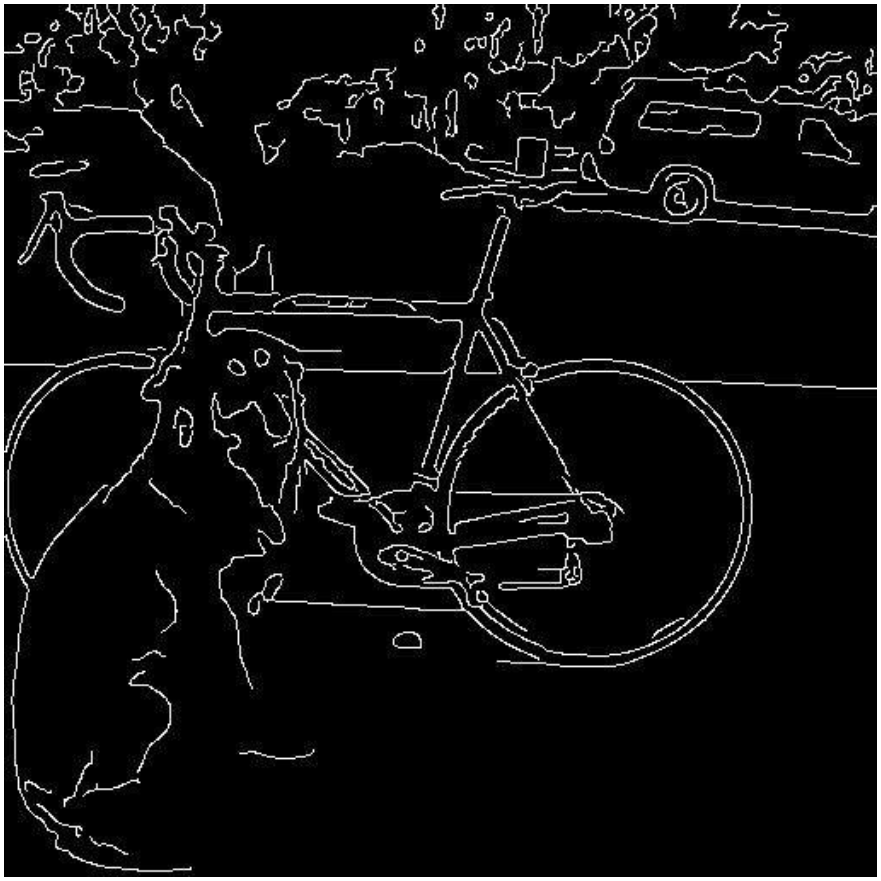
(b) Mark as valid edge pixels all the weak pixels in $g_{NL}(x, y)$ that are connected to p using, say, 8-connectivity.

(c) If all nonzero pixels in $g_{NH}(x, y)$ have been visited go to Step (d). Else, return to Step (a).

(d) Set to zero all pixels in $g_{NL}(x, y)$ that were not marked as valid edge pixels.

Step -4: Double Thresholding and Edge Linking

Before Edge Linking



After Edge Linking

Step -4: Double Thresholding and Edge Linking

Summary of Canny Edge Detector

(1) Filter image with derivative of Gaussian

(2) Find magnitude and orientation of gradient

(3) Non-maximum suppression

(4) Linking and thresholding (hysteresis):

Define two thresholds: low and high

Use the high threshold to start edge curves and the low threshold to continue them

Summary of Canny Edge Detector

J. Canny, ***A Computational Approach To Edge Detection***, IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Our first computer vision pipeline!
Still a widely used edge detector in computer vision

Summary of Canny Edge Detector

original Canny with $\sigma = 1$ Canny with $\sigma = 2$

- The choice of σ depends on desired behavior
  - large σ detects "large-scale" edges
  - small σ detects fine edges

- Readings:
  (1) Hough Transform - Page 737 - 742 - Digital Image Processing, 4th Ed, Rafael C. Gonzalez & Richard E. Woods

- Next Class:

  Image Sharpening; Linear Filters; Edge Detection using Canny; Line detection using Hough transforms; [if possible]

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Thank you